Climate Change Data Analysis Report

Author: Sean Travis
Date: 12/03/2026
Dataset: owid_climate_data.csv, cleaned_owid_climate_data.csv
Python File: climate_analysis.ipynb

1. Overview

Objective:
This report examines long-term climate trends by analysing countries CO_2 emissions per capita alongside their annual temperature anomalies over a period ranging from 1940-2023 where applicable. The dataset owid_climate_data.csv was compiled from CO_2 emissions per capita data and the Global temperature anomalies by month, which was then averaged to annual temperatures anomalies, this data was taken from Our Word in Data.

Note: AI was used to generate comments for the python code.

Dataset Summary:
Provide a top-level summary of the dataset.

- CO_2 : CO_2 emissions per country per year (metric tons).
- Temperature: Annual Temperature Anomaly (°C).
- Country: Country Name.
- Country Code: Code to find the country.
- Year: Year the data is from.

2. Step-by-Step Data Analysis Pipeline

Step 1: Data Import & Cleaning

What was done:

- Imported CSV file with pandas
- Checked how the data was structured and if any data was missing.

- Cleaned data removing any null data from annual temperature anomaly to ensure that data would be clean and avoid any inaccurate data further on.

Code Snippet:

```
# Import climate data and run initial data-quality checks.

import pandas as pd

# Load the source climate dataset.

df = pd.read_csv("owid_climate_data.csv")

# Inspect column types and non-null counts.

df.info()

print("\n")

# Report missing values before cleaning.

print("Data missing values\n")

print(df.isnull().sum())

# Remove rows missing CO2 emissions or temperature anomaly values.

df_cleaned = df.dropna(subset=["CO2 Emissions Per Capita", "Annual Temperature
Anomaly"])

print("Data missing values in CO2 Emissions Per Capita and Annual Temperature Anomaly")

print(df_cleaned.isnull().sum())

print("\nRows after cleaning:", len(df_cleaned), "rows removed:", len(df) - len(df_cleaned))

print("\nRows removed due to missing CO2 Emissions Per Capita or Annual Temperature
Anomaly to clean the dataset and avoid inaccurate data.")

# Save to the cleaned-data filename, then reload for downstream analysis.

df.to_csv('cleaned_owid_climate_data.csv', index=False)

df = pd.read_csv("cleaned_owid_climate_data.csv")

df.head()
```

Screenshot / Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 26182 entries, 0 to 26181
Data columns (total 5 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Country Name                26182 non-null  str
 1   Country Code                22895 non-null  str
 2   Year                        26182 non-null  int64
 3   CO2 Emissions Per Capita    26182 non-null  float64
 4   Annual Temperature Anomaly  14319 non-null  float64
dtypes: float64(2), int64(1), str(2)
memory usage: 1022.9 KB


Data missing values
Country Name                      0
Country Code                   3287
Year                              0
CO2 Emissions Per Capita          0
Annual Temperature Anomaly    11863
dtype: int64
Data missing values in Annual Temperature Anomaly
Country Name                  0
Country Code                  0
Year                          0
CO2 Emissions Per Capita      0
Annual Temperature Anomaly    0
dtype: int64
Rows after cleaning: 14319 rows removed: 11863
Rows removed due to missing Annual Termperature Anomaly to clean the dataset and avoid inaccurate data.
```

	Country Name	Country Code	Year	CO2 Emissions Per Capita	Annual Temperature Anomaly
0	Afghanistan	AFG	1949	0.001992	-2.020418
1	Afghanistan	AFG	1950	0.010837	-2.298886
2	Afghanistan	AFG	1951	0.011625	-1.287201
3	Afghanistan	AFG	1952	0.011468	-1.237078
4	Afghanistan	AFG	1953	0.013123	-1.084867

Fig 1. Data structure, missing data values, re-check after cleaning.

Summary:

This shows that the data has 11863 rows of missing data from annual temperature anomaly before being cleaned and after being cleaned there is no data missing. We can take from this that all country code missing data was also missing annual temperature anomaly. This will give us a clean dataset to work from avoiding inaccurate data form being presented.

Step 2: Exploring Sets & Writing Functions

What was done:

- Created a set of all unique countries.
- Defined a function to extract data for a given country and year range.

Code Snippet:

```
# Examine dataset scope and define a helper function to filter by country and year range.

# Count unique countries in the cleaned dataset.

country_set = set(df['Country Name'])

print("Countries in dataset:", len(country_set))

print("Original cleaned dataset has",len(df_cleaned), "country entries.")

# Define a helper to view one country's data within a year range.

def get_country_data(country, start_year, end_year):

    # Filter to the requested country, then constrain by the year bounds.

    country_data = df[df['Country Name'] == country]

    country_data = country_data[country_data['Year'] >= start_year]

  country_data = country_data[country_data['Year'] <= end_year]

    return country_data

# Example query.

get_country_data('India', 2000, 2020)
```

Screenshot / Output:

```
Countries in dataset: 187
Original cleaned dataset has 14319 country entires.
```

	Country Name	Country Code	Year	CO2 Emissions Per Capita	Annual Temperature Anomaly
6145	India	IND	2000	0.932435	-0.224110
6146	India	IND	2001	0.928220	-0.147684
6147	India	IND	2002	0.940360	0.215657
6148	India	IND	2003	0.956066	-0.042418
6149	India	IND	2004	0.998095	0.001837
6150	India	IND	2005	1.034569	-0.076595
6151	India	IND	2006	1.101976	0.174562
6152	India	IND	2007	1.169509	0.052127
6153	India	IND	2008	1.233049	-0.179187
6154	India	IND	2009	1.315532	0.537815
6155	India	IND	2010	1.348904	0.492528
6156	India	IND	2011	1.399205	-0.155027
6157	India	IND	2012	1.506013	-0.045455
6158	India	IND	2013	1.539630	-0.232880
6159	India	IND	2014	1.637111	-0.003478
6160	India	IND	2015	1.682363	0.275207
6161	India	IND	2016	1.752050	0.656726
6162	India	IND	2017	1.784719	0.518033
6163	India	IND	2018	1.886328	0.321343
6164	India	IND	2019	1.881088	0.284416
6165	India	IND	2020	1.726452	-0.003115

Fig 2. Total unique countries in the dataset and the data for India from 2000-2020.

Summary:

Able to create a set of all unique countries showing that there are 187 unique countries in the dataset. The function then ran correctly showing the data for India from 2000 to 2020. We can see from this data that CO_2 Emissions Per Capita have risen over the 20 years.

Step 3: Logical Filtering

What was done:

- Filtered all countries where CO_2 emissions were above 10 metric tons in 2020.

- Displayed only the Country Name, CO_2 Emissions Per Capita, Annual Temperature Anomaly columns.

Code Snippet:

```
# Select countries in 2020 with CO2 emissions per capita above 10 tonnes.

# Keep only columns needed for quick inspection.

filtered = df[(df['Year'] == 2020) & (df['CO2 Emissions Per Capita'] > 10)][['Country Name',
'CO2 Emissions Per Capita', 'Annual Temperature Anomaly']]

filtered
```

Screenshot / Output:

	Country Name	CO2 Emissions Per Capita	Annual Temperature Anomaly
707	Australia	15.506062	0.413426
1033	Bahrain	25.215979	0.544875
1867	Brunei	24.939121	0.488073
2326	Canada	13.795024	0.144869
4502	Faroe Islands	14.426020	0.148138
6985	Kazakhstan	13.113933	0.940694
7230	Kuwait	21.739220	0.725706
7896	Luxembourg	12.806264	1.281792
8578	Mongolia	11.265909	0.466132
9177	New Caledonia	17.981370	0.353113
9813	Oman	15.174401	0.060757
10575	Qatar	36.564110	0.610190
10743	Russia	11.156080	2.247584
11166	Saudi Arabia	19.879406	0.557455
11938	South Korea	11.524341	0.439023
12967	Trinidad and Tobago	23.677786	0.303785
13442	United Arab Emirates	23.006842	0.546586
13610	United States	13.889589	0.681033

Fig 3. Countries in 2020 with CO_2 Emissions Per Capita over 10 tonnes.

Summary:

This data shows that there are 18 countries with a CO_2 emission Per Capita over 10 tonnes for the year 2020.

Step 4: Probability & Summary Statistics

What was done:

- Calculated the min, max and mean of annual temperature anomaly for each year.
- Estimated the probability as a percentage that a recorded temperature exceeds 1.5°C.

Code Snippet:

```
# Summarize annual temperature anomaly distribution by year.

yearly_stats = df.groupby('Year')["Annual Temperature Anomaly"].agg(['min', 'max', 'mean'])

display(yearly_stats)

# Estimate the chance that a record exceeds 1.5 degrees C anomaly.

exceeding = df[df['Annual Temperature Anomaly'] > 1.5]

probability = len(exceeding) / len(df)

probability_percentage = probability * 100

print(f"Estimated probability that a recorded temperature will exceed 1.5°C:
{probability_percentage:.2f}%")
```

Screenshot / Output:

	min	max	mean
Year			
1940	-3.825056	0.423903	-1.624415
1941	-3.941369	1.779448	-1.251653
1942	-3.628582	0.683115	-1.368997
1943	-2.985201	1.080043	-1.021218
1944	-2.997106	0.915558	-1.000902
...	...	...	...
2019	-0.226331	1.697816	0.601391
2020	-0.231912	2.247584	0.580830
2021	-0.655329	1.511456	0.411804
2022	-0.582847	1.773571	0.444629
2023	-0.266950	1.894608	0.862665

84 rows × 3 columns

```
Estimated probability that a recorded temperature will exceed 1.5°C: 0.34%
```

Fig 4. Min, Max and Mean for each year, Probability estimate.

Summary:

From this data we can see that there is a 0.34% probability that one of the recorded temperatures will exceed 1.5°C.

Step 5: Counting Principles

What was done:

- Counted the total unique Year-Country Name combinations that exist in the dataset.

Code Snippet:

```
# Count distinct country-year pairs in the cleaned dataset.

unique_combos = df.groupby(['Country Name', 'Year']).size()

print("Number of unique combinations:", len(unique_combos))

print("Number of rows in the cleaned dataset:", len(df))

print("This count matches the number of rows in the cleaned dataset, confirming each row
represents a unique country-year record.")
```

Screenshot / Output:

```
Number of unique combinations: 14319
Number of rows in the cleaned dataset: 14319
This count matches the number of rows in the cleaned dataset, confirming each row represents a unique country-year record.
```

Fig 5. Shows total number of unique combinations.

Summary:

From this data we can see that the total unique combinations match the numbers of rows of data in the cleaned data set confirming each row of data stands for a unique country-year combination.

Step 6: Graph Theory

What was done:

- For the year 2020 created a graph where the countries are nodes.
- Drew edges between Countries if their CO_2 emissions differed by less than 1 tonne.

- Visualised the graph using network.

Code Snippet:

```
# Build a spring-layout network of countries with similar 2020 CO2 emissions.
import networkx as nx
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
# Keep only 2020 country and emissions values.
df_2020 = df[df['Year'] == 2020][['Country Name', 'CO2 Emissions Per Capita']]
# Create a graph where each country is a node with its CO2 value.
G = nx.Graph()
for _, row in df_2020.iterrows():
    G.add_node(row['Country Name'], co2=row['CO2 Emissions Per Capita'])
# Colour nodes by emissions for easier visual comparison.
co2_values = [G.nodes[node]['co2'] for node in G.nodes()]
node_colors = ['red' if G.nodes[node]['co2'] > 20
               else 'orange' if G.nodes[node]['co2'] > 10
               else 'green' for node in G.nodes()]
# Build legend entries for the node colours.
legend = [
    Patch(facecolor='red', label='CO2 > 20 tonnes per capita'),
    Patch(facecolor='orange', label='CO2 > 10 tonnes per capita'),
    Patch(facecolor='green', label='CO2 <= 10 tonnes per capita'),
]
# Connect countries whose emissions differ by less than 1.0 tonne.
```

```
countries = df_2020['Country Name'].tolist()

values = df_2020['CO2 Emissions Per Capita'].tolist()

for i in range(len(countries)):

    for j in range(i + 1, len(countries)):

        diff = abs(values[i] - values[j])

        if diff < 1.0:

            weight = 1 - diff

            G.add_edge(countries[i], countries[j], weight=weight)

# Draw graph with weighted edge labels.

plt.figure(figsize=(18, 16))

pos = nx.spring_layout(G, seed=100, k=1)

nx.draw(G, pos, with_labels=True, node_size=400, font_size=7,
node_color=node_colors, edge_color='lightblue')

plt.title("CO₂ Similarity Graph", fontsize=16)

plt.legend(handles=legend, loc='upper right', fontsize=18)

plt.show()
```

Screenshot / Output:

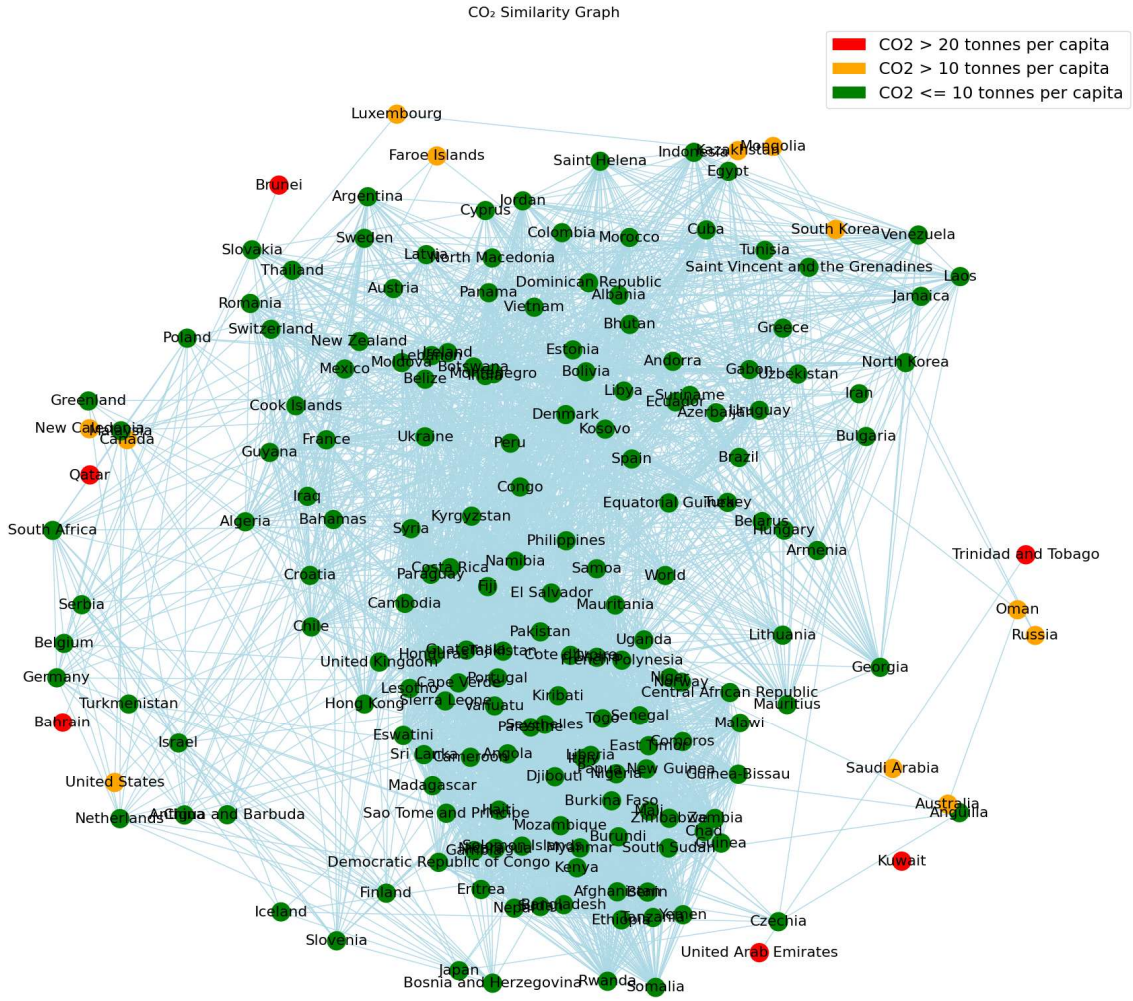

Fig 6. Spring layout graph showing CO_2 Similarities.

Summary:

From the data displayed in the graph we can see that the low emission countries are close together in their total emissions this is seen by the heavy density of edges around them. It also shows that Kuwait and Qatar our outliers with no countries close to them in emissions.

Step 7: Aggregation & Visualisation

What was done:

- Grouped data by year and calculated the average global temperature and displayed the results as a line graph.
- Created a bar chart displaying the total CO_2 emissions for each country for 2020.
- Created a scatterplot chart comparing CO_2 emissions and temperatures for 2020.

Code Snippet:

```
# Plot the yearly mean of annual temperature anomaly as a line chart.

# Keep 2020 subset available for any follow-up checks in this section.

df_2020 = df[df['Year'] == 2020]

# Aggregate by year and render with markers for each point.

df.groupby('Year')['Annual Temperature Anomaly'].mean().plot(kind='line',

marker='D',

markersize=5,

color='blue',

title='Average Temperature Over Time',

xlabel='Year',

ylabel='Annual Temperature Anomaly (°C)',

grid='show',

figsize=(10, 6),

)

plt.show()

# Compare 2020 emissions and temperature anomaly with a labelled scatter plot.

# Filter records to 2020 before plotting.
```

```
df_2020 = df[df['Year'] == 2020]

plt.figure(figsize=(10, 6))

plt.scatter(df_2020['CO2 Emissions Per Capita'], df_2020['Annual Temperature
Anomaly'])

plt.title('Emissions Per Capita vs Annual Temperature Anomaly for 2020')

plt.xlabel('CO2 Emissions per Capita (Tonnes)')

plt.ylabel('Annual Temperature Anomaly (°C)')

plt.grid(True)

# Add country labels to each point for direct identification.

for i, row in df_2020.iterrows():

    plt.text(row['CO2 Emissions Per Capita'], row['Annual Temperature Anomaly'],
row['Country Name'], fontsize=8)

plt.tight_layout()

plt.show()

# Build a bar chart of 2020 CO2 emissions per capita by country.

# Filter records to the 2020 snapshot.

df_2020 = df[df['Year'] == 2020]

# Plot country-level emissions and format labels for readability.

plt.figure(figsize=(24, 9))

plt.bar(df_2020['Country Name'], df_2020['CO2 Emissions Per Capita'])

plt.xticks(rotation=90)

plt.xlabel('Country')

plt.ylabel('CO2 Emissions Per Capita')

plt.title('CO2 Emissions per Capita 2020')

plt.show()
```

Screenshot / Output:

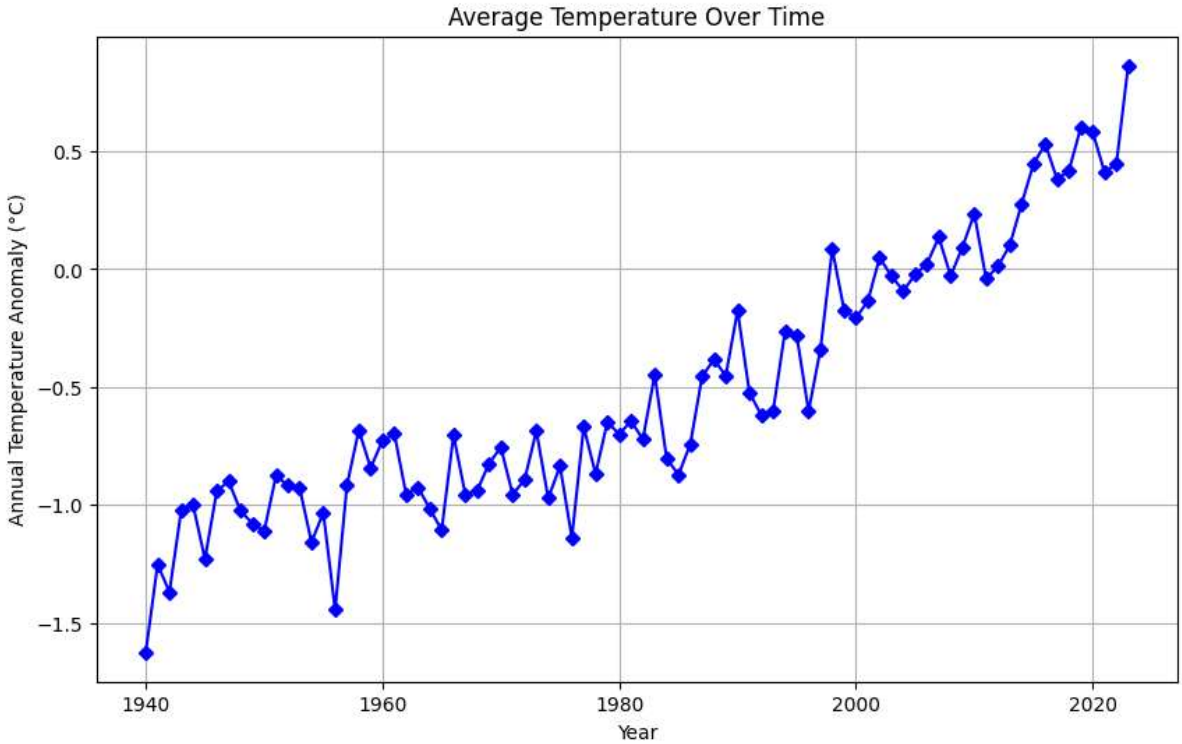

Fig 7. A line graph displaying the average temperature from 1940-2023

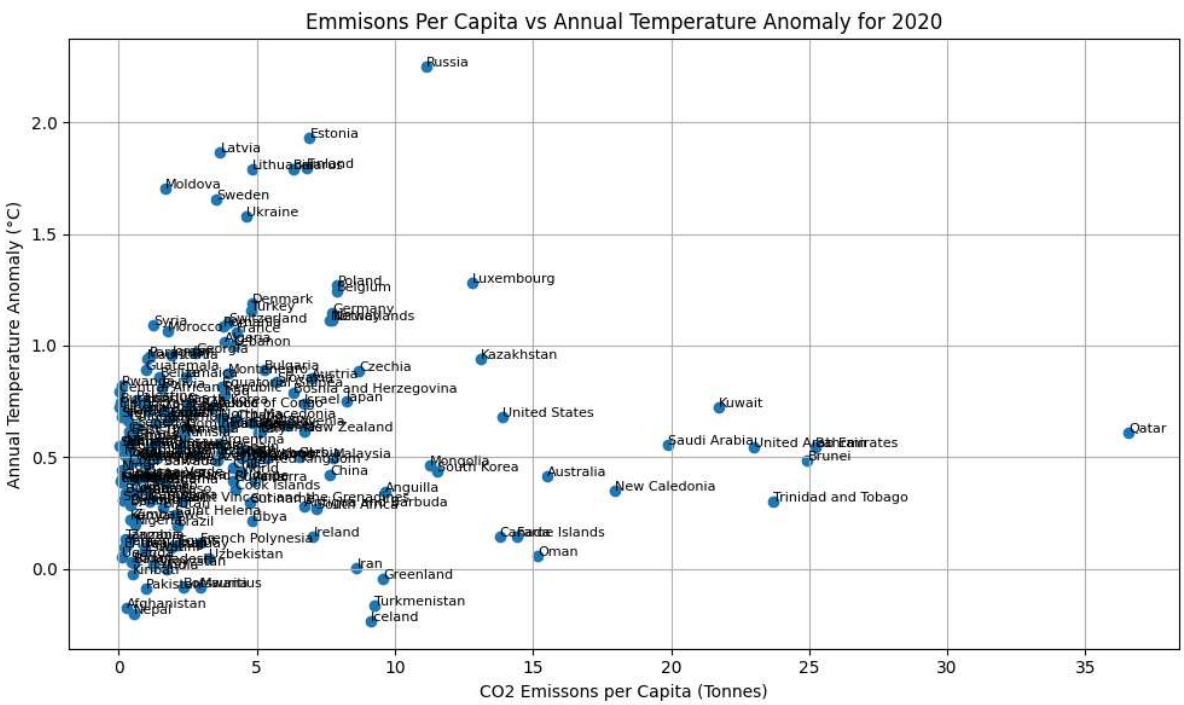

Fig 8. Scatterplot graph comparinng CO_2 emissions and Temperature anomalys for 2020.

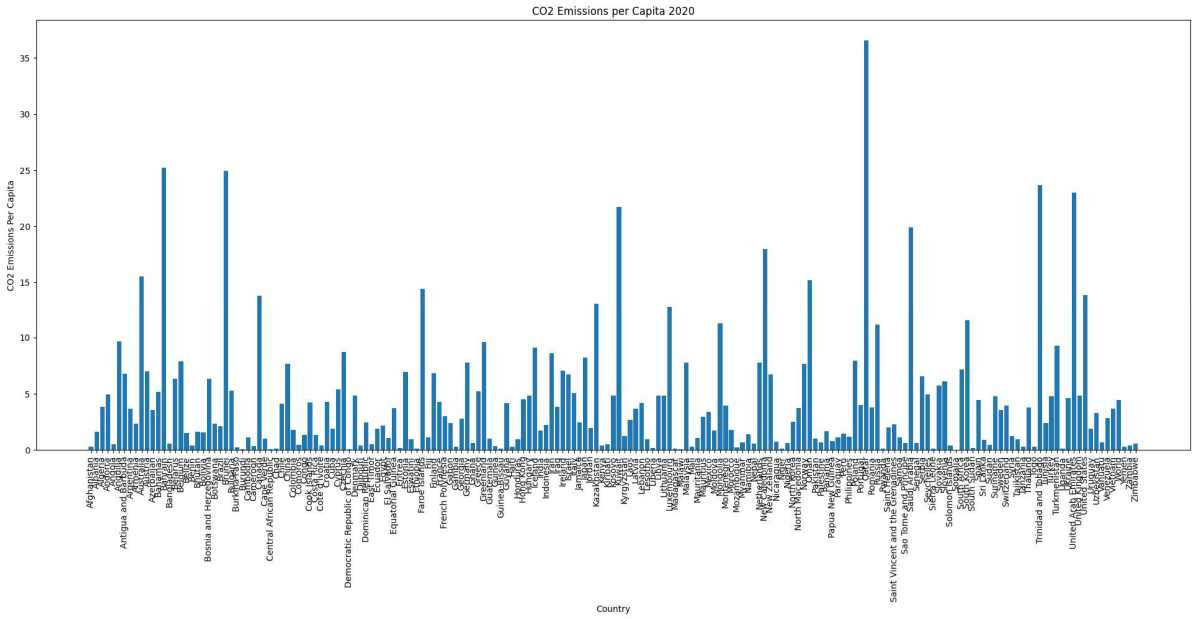

Fig 9. Bar Chart displaying the total CO_2 Emissions per country for 2020.

Summary:

From fig 7, we can see that the data shows the temperature has risen over the years.

From fig 8, we can see that the data shows that for the year 2020 most countries are grouped close together showing similar emissions and temperature anomalies with a couple of outliers like Russia and Qatar.

From fig 9, we can see that the data shows that for the year 2020 most countries had a CO_2 emissions below 10 tonnes with Qatar being an outlier with emissions over 35 tonnes.

3. Summary of Findings

Key Observations:

From the dataset provided we have found that there is a total of 11863 rows of missing data from annual temperature anomaly, once cleaned we are left with 14319 rows of data to use with a total of 187 unique countries in the dataset. Furthermore, there is a total of 14319 combinations of country-year data entries in the dataset.

From the data we can see that for the year of 2020 CO_2 emissions per capita varied widely between countries but with a majority being under 10 tonnes with several outliers. The data also shows a rise in average temperature over the years. We have also worked out that there is a 0.35% probability that a selected data entry will have a temperature anomaly of 1.5°C.

4. Appendix

Note: AI was used to generate comments for the python code, this was done after all code was written and checked to make sure outputs where as desired. The prompt used was add comments to all cells.

Step 1. Changes made

Here are the code and output for removing CO_2 Emissions and Temperature Anomalies, I did not use this in the report as I checked the data first for what data was missing and there was no missing CO_2 Emission Data only Annual Temperature Anomaly and Country Code.

Code snippet:

```
# Remove rows missing CO2 emissions or temperature anomaly values.

df_cleaned = df.dropna(subset=["CO2 Emissions Per Capita", "Annual Temperature Anomaly"])

print("Data missing values in CO2 Emissions Per Capita and Annual Temperature Anomaly")

print(df_cleaned.isnull().sum())

print("Rows after cleaning:", len(df_cleaned), "rows removed:", len(df) - len(df_cleaned))

print("Rows removed due to missing CO2 Emissions Per Capita or Annual Temperature Anomaly to clean the dataset and avoid inaccurate data.")

df.to_csv('cleaned_owid_climate_data.csv', index=False)

df.head()
```

```
Data missing values in CO2 Emissions Per Capita and Annual Temperature Anomaly
Country Name                  0
Country Code                  0
Year                          0
CO2 Emissions Per Capita      0
Annual Temperature Anomaly    0
dtype: int64
Rows after cleaning: 14319 rows removed: 11863
Rows removed due to missing CO2 Emissions Per Capita or Annual Termperature Anomaly to clean the dataset and avoid inaccurate data.
```

	Country Name	Country Code	Year	CO2 Emissions Per Capita	Annual Temperature Anomaly
0	Afghanistan	AFG	1949	0.001992	-2.020418
1	Afghanistan	AFG	1950	0.010837	-2.298886
2	Afghanistan	AFG	1951	0.011625	-1.287201
3	Afghanistan	AFG	1952	0.011468	-1.237078
4	Afghanistan	AFG	1953	0.013123	-1.084867

Fig.# Shows the output is still the same even when removing CO_2 Emissions Per Capita

Included Files:

- Python Script: climate_analysis.ipynb
- Report: CST184_Project2_Sean_Travis_1035888.pdf